

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY DESIGN AND MANUFACTURING KURNOOL

Department of Computer Science and Engineering

MID-SEMESTER PROJECT REPORT

Composing Context Compression for Long-Horizon LLM Agents

*Controlled Composition and Critical-Fact Retention
on top of ACON*

A report submitted in partial fulfilment of the requirements
for the award of the degree of

Bachelor of Technology

Computer Science and Engineering

Submitted by

[ROLL NO. 1] [STUDENT NAME 1]

[ROLL NO. 2] [STUDENT NAME 2]

[ROLL NO. 3] [STUDENT NAME 3]

Project Supervisor

[SUPERVISOR NAME]

Department of Computer Science and Engineering
Indian Institute of Information Technology Design and Manufacturing Kurnool
Kurnool, Andhra Pradesh – 518008

September 2026

1. Introduction

LLM agents solve long-horizon tasks by alternating actions and observations over tens of steps, and every tool output and intermediate thought is appended to the prompt. The context therefore grows without bound. This causes two problems: inference memory grows with the KV cache, and long, partly stale contexts distract the model and degrade its decisions [1]. Losing a single file path or API token can derail an entire trajectory, so compression must be aggressive and precise at the same time.

Existing methods compress one kind of context at a time: the interaction history, the latest observation, or a document. Agents need several of these stages at once, and it is not known how they interact. This project takes ACON [1] as the base method. We reproduce it and SAC [2] on our own hardware, and study how compression stages should be *composed* and *evaluated*.

2. Literature Review

Generative, prompt-level compression. ACON [1] uses an LLM to rewrite the history once it exceeds a threshold T_{hist} and to shorten observations longer than T_{obs} . The compression guideline is optimised in natural language: trajectories that succeed without compression but fail with it are contrasted to produce feedback, and the compressor is then distilled into smaller models. On AppWorld [7] with gpt-4.1, history compression reaches 56.5% accuracy against 56.0% with no compression, and peak tokens drop by 26–54% across three benchmarks.

Soft, KV-level compression. SAC [2] selects anchor tokens from the context, marks them with a learned embedding, and uses bidirectional attention so that their key-value pairs absorb the whole context, without autoencoding pretraining. On MRQA with Llama-3.2-1B at $15\times$ compression, it reaches 54.95 in-domain F1, above EPL (51.52).

Extractive, code-specific pruning. SWE-Pruner [3] places a 0.6B line-level classifier between a coding agent and its tools. The agent writes a goal hint that conditions which lines of retrieved code are kept. On SWE-Bench Verified it cuts tokens by 23–38% while raising the resolve rate by 1.2–1.4 points.

Token pruning and RL-trained memory. LLMLingua [4] removes low-information tokens using a small language model. It is task-agnostic, and in ACON’s AppWorld comparison it reaches 39.3% accuracy for history compression. MEM1 [5] and MemAgent [6] instead train the agent with RL to maintain a bounded memory. MemAgent extrapolates from an 8K window to 3.5M-token QA with under 10% loss, but these methods require updating the agent’s weights.

Property	ACON	SAC	SWE-Pruner	LLMLingua	MEM1/MemAgent	Proposed
Output form	Text	KV cache	Text lines	Text tokens	Text memory	Text
Agent weights frozen	Yes	Yes	Yes	Yes	No	Yes
History compression	Yes	No	No	Yes	Merged	Yes
Observation compression	Yes	No	Yes	Yes	Merged	Yes
Composition studied	Partly	No	No	No	No	Yes
Intrinsic quality metric	No	No	No	No	No	Yes (CFR)

Table 1: Comparison of context-compression families. “Partly”: ACON reports a combined setting in an appendix ablation only. “Merged”: one memory holds both.

3. Research Gaps

- Token savings do not become time savings.** ACON’s compressor generates autoregressively, and rewriting the history invalidates the agent’s KV cache, forcing the compressed prefix to be recomputed. ACON’s own measurement on AppWorld (Qwen3-14B agent and compressor, one A100) gives a median of 73.24s per task without compression, 87.68s with history compression and 101.92s with observation compression [1].
- The combined setting is schedule-confounded.** History compression fires when $|h_t| > T_{\text{hist}}$. Once observations are compressed first, $|h_t|$ grows more slowly, so history compression fires at different steps and on already-pruned text. ACON’s combined history+observation setting (45.8% accuracy, versus 56.5% and 53.6% for history and observation compression alone [1]) therefore changes two things at once and is not a clean composition of two treatments.
- Compression quality is only measured indirectly.** ACON and SAC judge a compressor solely by downstream success or F1/EM. Neither measures whether the specific facts the agent will need survive compression, so a failure cannot be attributed to information loss rather than to the agent.

4. Problem Statement

Given a long-horizon agent with frozen weights, a history compressor and an observation compressor, determine how the two stages should be composed and scheduled so that the token savings of each are retained without

lowering task success. The comparison between single-stage and composed settings must hold the compression schedule fixed, and compression quality must be measured directly, as the retention of task-critical state, rather than inferred only from task success.

5. Proposed Solution

We propose three changes to ACON’s pipeline (Figure 1).

1. Decoupled composition order. The observation compressor still produces a short o'_t for the agent’s immediate step, but the raw o_t is retained in a side buffer. When history compression fires, it summarises the raw observations rather than the already-pruned ones, so information discarded by the observation stage can still enter the history summary.

2. Controlled compression schedule. The trigger steps recorded in the history-only run are replayed unchanged in the composed run, instead of recomputing $|h_t| > T_{\text{hist}}$ on pruned text. Single-stage and composed conditions then differ only in the treatment, which removes the confound of gap (b).

3. Critical-Fact Retention (CFR). From each uncompressed trajectory we extract state variables the agent later uses: access tokens, entity IDs, file paths and numeric values. With F the set of such facts present before a compression call and c the compressor output,

$$\text{CFR} = \frac{1}{|F|} \sum_{f \in F} \mathbf{1}[f \text{ appears verbatim in } c].$$

CFR is an exact-match survival rate that can be computed per compression call without rerunning the agent, and correlated with task success.

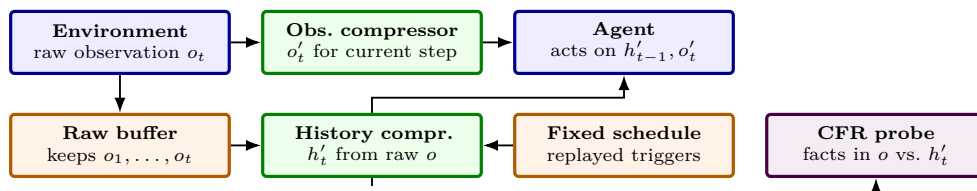


Figure 1: Proposed pipeline. The history compressor reads raw observations from a side buffer (orange), fires on a schedule fixed across conditions, and every call is scored by the CFR probe (violet).

6. Implementation and Results

Setup. All runs use one NVIDIA RTX 2000 Ada (16 GB). For ACON, models are served locally through Ollama: the agent is `qwen2.5:14b` and the compressor `qwen2.5:7b`. We use the AppWorld `train_history_tiny` split (38 tasks) with seed 42 and temperature 0. For SAC, we use the authors’ Llama-3.2-1B checkpoints on MRQA.

ACON reproduction (preliminary). Table 2 reports our AppWorld runs. The fast runs use 8 tasks with at most 12 iterations. The full history-compression run had completed 18 of 38 tasks at the time of writing, so it is compared with the baseline on the same 18 tasks. On these, history compression solved 9 tasks against 7 and used 24.5% fewer agent input tokens. The change is not uniform: it solved 5 tasks the baseline failed and failed 3 the baseline solved. On the 8-task fast runs, both compression settings finished slightly sooner than the baseline, so the latency overhead ACON reports (gap (a)) did not appear at this scale.

Run	Tasks	Max iter.	Solved	Avg iter.	Agent input tok.	Wall clock (s)
Baseline, no compression	8	12	1	11.00	269,935	953.9
History compression ($T_{\text{hist}} = 512$)	8	12	2	10.75	232,749	882.8
Observation compression ($T_{\text{obs}} = 256$)	8	12	1	11.00	261,422	873.6
Baseline, full split	38	30	12 (31.6%)	23.7	2,972,376	7312
Baseline, matched subset	18	30	7	22.22	1,283,906	not logged
History compression, matched subset	18	30	9	19.50	969,389	not logged

Table 2: Preliminary ACON runs on AppWorld (`qwen2.5:14b` agent, `qwen2.5:7b` compressor, seed 42). Wall-clock time is not logged for the matched subset because the baseline’s progress log is incomplete.

These numbers carry three caveats. First, Ollama served both models with a 4096-token context, so long baseline prompts were truncated and part of any gain may come from staying under this limit. Second, thresholds are 512/256 rather than the paper’s 4096/1024, because the paper values rarely fire on this split. Third, each setting was run with a single seed.

SAC reproduction. Evaluating the released checkpoints reproduces the published macro-averaged MRQA scores to within 0.2 F1: at $15\times$ we obtain 54.98 in-domain / 39.25 out-of-domain F1 (published 54.95 / 39.26), and at $5\times$ 63.70 / 47.71 (published 63.63 / 47.72).

SAC $\times$ ACON pilot (preliminary). We trained two ACON-inspired variants of SAC for 5,000 steps at $15\times$ (single seed, evaluated on 6,786 examples) against a matched SAC control (51.37 / 38.62 F1). Failure-gated distillation, which distills from the full-context model only where compression fails, gave -0.23 in-domain F1 (95% CI $[-0.95, +0.47]$), i.e. no effect. Task-guided anchors combined with failure-gated distillation reached 60.03 / 44.71 F1, a gain of $+8.66$ [$+7.66, +9.60$] in-domain. That variant sees the question during compression, so it is not directly comparable to question-agnostic SAC, and task-guided anchors alone have not yet been run.

7. Conclusion

We have reproduced SAC’s published results and set up a local ACON pipeline on AppWorld. On the 18 tasks run so far, history compression solves more tasks with 24.5% fewer agent input tokens. We identified three gaps in how context compressors are composed and evaluated: latency, schedule confounding and the lack of an intrinsic quality metric. Next, we will complete the 38-task history and observation runs and add runs at the paper thresholds. We will then implement the raw-observation buffer, the fixed schedule and the CFR metric, and evaluate the composed pipeline across multiple seeds.

References

- [1] M. Kang, W.-N. Chen, D. Han, H. A. Inan, L. Wutschitz, Y. Chen, R. Sim, and S. Rajmohan, “ACON: Optimizing Context Compression for Long-horizon LLM Agents,” in *Proc. 43rd Int. Conf. Machine Learning (ICML)*, PMLR 306, 2026.
- [2] X. Liu, R. Zhao, P. Huang, X. Liu, J. Xiao, C. Xiao, T. Xiao, S. Gao, Z. Yu, and J. Zhu, “Autoencoding-Free Context Compression for LLMs via Contextual Semantic Anchors,” in *Int. Conf. Learning Representations (ICLR)*, 2026.
- [3] Y. Wang, Y. Shi, M. Yang, R. Zhang, S. He, H. Lian, Y. Chen, S. Ye, K. Cai, and X. Gu, “SWE-Pruner: Self-Adaptive Context Pruning for Coding Agents,” arXiv:2601.16746, 2026.
- [4] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, “LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models,” in *Proc. EMNLP*, pp. 13358–13376, 2023.
- [5] Z. Zhou, A. Qu, Z. Wu, S. Kim, A. Prakash, D. Rus, J. Zhao, B. K. H. Low, and P. P. Liang, “MEM1: Learning to Synergize Memory and Reasoning for Efficient Long-Horizon Agents,” arXiv:2506.15841, 2025.
- [6] H. Yu, T. Chen, J. Feng, J. Chen, W. Dai, Q. Yu, Y.-Q. Zhang, W.-Y. Ma, J. Liu, M. Wang, and H. Zhou, “MemAgent: Reshaping Long-Context LLM with Multi-Conv RL-based Memory Agent,” in *Int. Conf. Learning Representations (ICLR)*, 2026.
- [7] H. Trivedi, T. Khot, M. Hartmann, R. Manku, V. Dong, E. Li, S. Gupta, A. Sabharwal, and N. Balasubramanian, “AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents,” in *Proc. ACL*, pp. 16022–16076, 2024.